EATA: Explainable Algorithmic Trading Agent via Neural-Guided Symbolic Regression

Anonymous Authors Anonymous Institution

Abstract—Algorithmic trading systems powered by deep learning have achieved remarkable performance but suffer from a critical limitation: lack of interpretability. This opacity hinders trust, regulatory compliance, and strategy refinement. We propose EATA (Explainable Algorithmic Trading Agent), a novel framework that combines Monte Carlo Tree Search (MCTS) with neural network guidance to automatically discover interpretable mathematical expressions for stock price prediction. Our key contributions include: (1) a three-headed Policy-Value-Profit network (PVNet) that jointly optimizes for search guidance, prediction accuracy, and trading profitability; (2) a domain-specific grammar incorporating financial operators such as moving averages, momentum, and volatility indicators; (3) a grammar augmentation mechanism that extracts and reuses high-quality sub-expressions to accelerate search; and (4) a sliding window training scheme with expression inheritance for temporal adaptation. Extensive experiments on 500 stocks over a 20-year period demonstrate that EATA achieves competitive returns compared to state-of-the-art deep learning methods while providing fully interpretable trading rules in the form of mathematical formulas. The discovered expressions reveal meaningful market patterns and can be directly validated by domain experts.

Index Terms—Algorithmic Trading, Symbolic Regression, Monte Carlo Tree Search, Explainable AI, Neural-Guided Search

1 Introduction

Algorithmic trading systems powered by deep learning have achieved remarkable performance in financial markets [1], [2]. However, these "black-box" models present a fundamental challenge: their decision-making processes remain opaque, hindering trust, regulatory compliance (e.g., MiFID II [3]), and strategic refinement. This opacity is particularly problematic in finance, where understanding *why* a model fails is crucial for risk management.

Symbolic regression offers a promising alternative by discovering interpretable mathematical expressions that map inputs to outputs [4], [5]. Unlike neural networks, symbolic expressions can be directly validated by domain experts and audited by regulators. However, existing symbolic regression approaches face a critical limitation when applied to trading: **they typically optimize for point-wise prediction accuracy (e.g., MSE), which does not necessarily correlate with trading profitability**.

Consider the following fundamental disconnect: minimizing MSE penalizes all prediction errors equally, regardless of their economic impact. In trading, a small error in predicting a large directional move (strong trend) can be far more costly than a large error during sideways markets. Moreover, financial returns exhibit heavy-tailed distributions with significant skewness and kurtosis [6]—properties that MSE-based objectives fail to capture.

This observation leads to our **central research question**: *How can we design a symbolic regression framework that explicitly optimizes for trading profitability rather than simple prediction accuracy?* Answering this requires addressing three sub-questions:

- RQ1 **Objective Design**: What distributional divergence metric can effectively capture the asymmetric, heavy-tailed nature of financial returns?
- RQ2 **Search Guidance**: How can neural networks guide symbolic search to discover profitable expressions without sacrificing interpretability?
- RQ3 **Empirical Validation**: Can profit-optimized symbolic models achieve risk-adjusted returns competitive with black-box deep learning?

Our key insight is that the **Wasserstein distance** (Earth Mover's Distance) provides a theoretically grounded measure for comparing return distributions that respects both tail risk and directional asymmetry. Unlike KL divergence or MSE, Wasserstein distance measures the minimal "work" required to transform one distribution into another, making it sensitive to both the magnitude and location of distributional differences.

Based on this insight, we propose **EATA** (**E**xplainable **A**lgorithmic **T**rading **A**gent), a neural-guided symbolic regression framework with three key innovations:

1) **Profit-Aware Learning**: A three-headed neural network (PVNet) with policy, value, and *profit* heads. The profit head explicitly predicts the Wasserstein-based trading reward, guiding MCTS toward expressions with favorable risk-return profiles.
2) **Domain-Specific Grammar**: A context-free grammar incorporating financial operators (moving averages, momentum, RSI, volatility) with fixed windows, encoding domain knowledge to constrain the search space.
3) **Temporal Adaptation**: A sliding window training scheme with expression inheritance and grammar augmentation, enabling adaptation to non-stationary market conditions.

We evaluate EATA on **500 high-volume stocks** from diverse sectors over a 4-year period (2020–2024), comparing against multiple baselines including state-of-the-art

deep learning (LSTM, Transformer), reinforcement learning (PPO), and symbolic regression (GP, NEMoTS). Results show that EATA achieves a Sharpe ratio of **0.83**, significantly outperforming the best baseline while providing fully interpretable trading rules.

Contributions. Our main contributions are:

1) We formalize the symbolic regression objective for trading as a **distribution matching problem** using Wasserstein distance, bridging the gap between prediction accuracy and economic utility.
2) We extend NEMoTS [7] with a **dedicated profit head** that predicts trading rewards, enabling neural-guided search to explicitly optimize for profitability.
3) We conduct **extensive empirical validation** on real-world market data with in-depth ablations, demonstrating that profit-aware symbolic regression can compete with black-box deep learning.

Paper Organization. Section 2 reviews related work. Section 3 presents EATA's methodology. Section 4 describes experiments. Section 5 concludes.

2 Related Work

This section reviews prior work in symbolic regression for trading, organized along two dimensions: (1) the search paradigm (evolutionary vs. neural-guided), and (2) the optimization objective (accuracy vs. profitability).

2.1 Symbolic Regression: Evolutionary vs. Neural-Guided

Evolutionary Approaches. Genetic Programming (GP) [4] has been the dominant paradigm for symbolic regression since the 1990s, evolving expression trees through selection, crossover, and mutation. Schmidt and Lipson [5] demonstrated GP's ability to rediscover physical laws. Modern implementations like gplearn [8] and Operon [9] improve efficiency through vectorized evaluation. However, GP suffers from bloat (uncontrolled growth) and premature convergence on rugged fitness landscapes [10].

Neural-Guided Approaches. Recent work leverages neural networks to guide symbolic search:

- **DSR** [11]: Uses an RNN as a policy network trained via REINFORCE to generate expressions token-by-token.
- **AI-Feynman** [12]: Combines neural fitting with physics-inspired techniques (dimensional analysis, symmetry detection).
- **Transformer-based** [13]: Maps input-output pairs directly to symbolic expressions.

2.2 Monte Carlo Tree Search for Symbolic Regression

Monte Carlo Tree Search (MCTS) [14] has emerged as a powerful alternative to GP, balancing exploration and exploitation through the UCB formula [15]. Key milestones include White et al. [16] applying MCTS to symbolic regression and Cazenave [17] introducing nested MCTS.

Most relevant to our work is **NEMoTS** (Neural Equation Monte Carlo Tree Search) [7], which combined MCTS with a Policy-Value network inspired by AlphaGo [18]. NEMoTS achieved state-of-the-art results on standard SR benchmarks.

EATA vs. NEMoTS. While EATA builds upon the NEMoTS framework, we introduce three critical innovations tailored for financial trading:

1) **Profit Head**: NEMoTS uses policy and value heads for search guidance and expression quality estimation (MSE-based). EATA adds a *third* head explicitly predicting trading profitability via Wasserstein distance.
2) **Domain Grammar**: NEMoTS uses generic mathematical operators. EATA introduces domain-specific financial operators (e.g., MA, RSI, Momentum) with fixed windows.
3) **Sliding Window**: EATA explicitly addresses non-stationarity through temporal adaptation and expression inheritance, whereas NEMoTS assumes a stationary data distribution.

Table 1 provides a detailed component-wise comparison.

TABLE 1: Detailed Comparison: EATA vs. NEMoTS

Component	NEMoTS	EATA (Ours)
Search Algorithm	MCTS + UCB	MCTS + UCB (same base)
Neural Guidance	Policy + Value heads	Policy + Value + **Profit** heads
Reward Signal	$r = 1/(1 + \text{MSE})$	$r^V = 1/(1 + \text{MSE})$, $r^P = 1/(1 + \mathcal{W}_1)$
Grammar	Generic math ops	Generic + **Financial ops**
Temporal Adaptation	None	**Sliding window** + inheritance
Grammar Augmentation	Yes	Yes (same mechanism)

2.3 Machine Learning for Algorithmic Trading

Deep learning approaches for trading include LSTM/GRU [1], [19] for capturing long-term dependencies and Transformers [20] for multi-scale pattern detection. Deep Reinforcement Learning (DRL) frameworks like FinRL [21] frame trading as sequential decision-making. While these methods achieve strong performance, they remain black-boxes, limiting regulatory compliance and human oversight [22].

2.4 Positioning of EATA

Table 2 compares EATA with related approaches across key dimensions. EATA uniquely combines neural-guided search, domain-specific financial operators, and profit-aware training within an interpretable symbolic regression framework.

TABLE 2: Comparison with Related Approaches

Method	Interp.	Neural	Finance	Profit	Stationarity
LSTM/Transformer	✗	✓	✗	✗	✓
FinRL (PPO)	✗	✓	✓	✓	✓
GP (Symbolic)	✓	✗	✗	✗	✗
Alpha Mining	✓	✗	✓	✗	✗
DSR	✓	✓	✗	✗	✗
NEMoTS	✓	✓	✗	✗	✗
EATA (Ours)	✓	✓	✓	✓	✓

3 Methodology

This section presents the technical details of the Explainable Algorithmic Trading Agent (EATA). We first establish a unified notation system summarized in Appendix A, then describe the core algorithms with their design rationale.

3.1 Problem Formulation

Given a multivariate financial time series $\mathbf{X} = \{\mathbf{x}_t\}_{t=1}^{T}$ where $\mathbf{x}_t \in \mathbb{R}^d$ represents d market features at time t, we seek an interpretable expression e^* that balances predictive accuracy with trading profitability:

$$e^* = \arg\max_{e \in \mathcal{E}} \left[\eta \cdot \mathcal{A}(e, \mathbf{X}) + (1-\eta) \cdot \mathcal{P}(e, \mathbf{X})\right], \tag{1}$$

where $\mathcal{E}$ denotes the valid expression space defined by grammar $\mathcal{G}$, $\mathcal{A}(\cdot)$ measures prediction accuracy, $\mathcal{P}(\cdot)$ measures trading profit, and $\eta \in [0, 1]$ balances the two objectives.

3.2 Domain-Specific Grammar Design

We employ a context-free grammar $\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$ tailored for financial time series analysis. The grammar incorporates domain knowledge through fixed-window financial operators.

Non-terminals and Terminals. The non-terminal set $\mathcal{V} = \{A, B, D\}$ includes primary expressions (A), helper expressions (B), and decision predicates (D). The terminal set Σ comprises feature variables $\{x_0, x_1, \ldots, x_{d-1}\}$, constants, and operators.

Production Rules. The rule set $\mathcal{R}$ comprises four categories:

Basic Arithmetic:

$$A \to A + A \mid A - A \mid A \times A \mid A \div A \mid A \times C \mid A + C \tag{2}$$

Mathematical Functions:

$$A \to \cos(A) \mid \sin(A) \mid \exp(A) \mid \log(B) \mid \sqrt{B} \mid |A| \mid A^2 \tag{3}$$

Financial Operators (Fixed-Window):

$$A \to \text{delay}(A, 1) \mid \text{delay}(A, 5) \mid \text{ma}(A, 5) \mid \text{ma}(A, 10) \mid \text{ma}(A, 20) \mid \text{diff}(A, 1) \mid \text{diff}(A, 5) \mid \text{mom}(A, 10) \mid \text{max_n}(A, 5) \mid \text{min_n}(A, 5) \mid \text{rsi}(A, 14) \mid \text{vol}(A, 20) \tag{4}$$

Conditional Logic:

$$A \to \text{ite}(D, A, A), \quad D \to A > B \mid A < B \tag{5}$$

Design Rationale. Fixed-window operators (e.g., $\text{ma}(\cdot, 10)$) rather than parameterized variants reduce the search space while preserving expressiveness. Predefined windows $(5, 10, 14, 20)$ align with standard technical analysis horizons. The conditional operator ite enables piecewise expressions for regime-switching strategies.

3.3 Neural-Guided Monte Carlo Tree Search

Algorithm 1 presents the MCTS procedure with neural network guidance. Each episode traverses the search tree using a policy fusion mechanism.

The UCB formula balances exploration and exploitation:

$$\text{UCB}(s, a) = \frac{Q(s, a)}{N(s, a)} + C\sqrt{\frac{\ln N(s)}{N(s, a)}} \tag{6}$$

Design Rationale. The dual fusion mechanism addresses uncertainty in neural network predictions early in training. When α is small (buffer not full), rollout provides reliable value estimates; as training progresses, α increases, trusting the neural network more. The value-profit fusion with ω ensures expressions are selected based on both predictive fit and profit potential.

Algorithm 1 Neural-Guided MCTS for Symbolic Regression

Require: Data $\mathbf{X}$, grammar $\mathcal{G}$, network f_θ, episodes K, guidance weight α, fusion weight ω, prior tree τ^{prev}
Ensure: Best expression e^*, best tree τ^*, experience buffer $\mathcal{D}$

```
Initialize Q(·) ← 0, N(·) ← 1, D ← ∅
s ← InitializeState(τ^prev)
for k = 1 to K do
    s ← s_0, states ← [], actions ← []
    U ← GetUnvisited(s)
    while U = ∅ and not IsTerminal(s) do        ▷ Selection
        π^nn, V, P ← f_θ(s, X)
        π^ucb ← ComputeUCB(s, C)
        π ← α · π^nn + (1 − α) · π^ucb
        π ← Normalize(π)
        a ∼ Categorical(π)
        states.append(s), actions.append(a)
        s ← Step(s, a)
        U ← GetUnvisited(s)
        if |s| ≥ L^max then
            Backprop(states, actions, 0)
            break
        end if
    end while
    if U ≠ ∅ then                               ▷ Expansion
        π^nn, V, P ← f_θ(s, X)
        a ∼ Categorical(α · π^nn + (1 − α) · Uniform(U))
        s' ← Step(s, a)
        r, e ← Evaluate(s', X)
        if IsTerminal(s') then
            UpdateModules(e, r)
        end if
    else                                        ▷ Simulation via leaf evaluation
        r, e ← Evaluate(s, X)
    end if
    v^nn ← ω · V + (1 − ω) · P                  ▷ Fuse value heads
    v^roll ← Rollout(s, n^play)                 ▷ Random simulations
    r^final ← α · v^nn + (1 − α) · v^roll       ▷ Fuse neural and rollout
    Backprop(states, actions, r^final)
    D.append((states, X, π, v^nn))
end for
return e*, τ*, D
```

3.4 Profit-Aware RL Reward via Wasserstein Distance

To explicitly optimize for trading profitability, we introduce a Reinforcement Learning (RL) reward signal r^{rl}. Unlike standard regression targets that penalize point-wise errors (e.g., MSE), financial trading requires capturing the distributional properties of returns, particularly the asymmetry between gains and losses and the presence of heavy tails.

Given predictions $\hat{\mathbf{y}}^{(1)}, \ldots, \hat{\mathbf{y}}^{(k)}$ from the top-k discovered expressions, we construct a prediction distribution $\mathcal{D}^{\text{pred}}$. We compare this against the distribution of actual future returns $\mathcal{D}^{\text{actual}}$ using the 1-Wasserstein distance (Earth Mover's Distance):

$$\mathcal{W}_1(\mathcal{D}^{\text{pred}}, \mathcal{D}^{\text{actual}}) = \int_{-\infty}^{\infty} |F^{\text{pred}}(x) - F^{\text{actual}}(x)|\, dx \tag{7}$$

where F^{pred} and F^{actual} are the cumulative distribution functions (CDFs) of the predicted and actual returns, respectively. The RL reward is then defined as:

$$r^{\text{rl}} = \frac{1}{1 + \mathcal{W}_1(\mathcal{D}^{\text{pred}}, \mathcal{D}^{\text{actual}})} \tag{8}$$

Why Wasserstein? Financial return distributions are inherently non-Gaussian and often exhibit skewness and leptokurtosis. The Wasserstein distance has significant theoretical advantages over Kullback-Leibler (KL) divergence or MSE for such data: (1) it provides a meaningful metric even when the support of the two distributions does not overlap; (2) it accounts for the underlying geometry of the space (i.e., the magnitude of errors matters, not just the probability mass difference); and (3) it effectively captures tail risks by measuring the global transport cost to map the predicted distribution to the actual one. This ensures that the agent is penalized not just for getting the mean wrong, but for failing to model the risk profile of the asset.

3.5 Three-Headed Policy-Value-Profit Network

The PVNet architecture (Algorithm 2) processes both the derivation state and input time series.

Algorithm 2 PVNet Forward Pass

Require: State sequence $\boldsymbol{\sigma} = [r_1, \ldots, r_l]$, time series $\mathbf{X} \in \mathbb{R}^{T \times d}$
Ensure: Policy $\boldsymbol{\pi}$, value V, profit P

```
E ← EMBEDDING(σ)                          ▷ Rule embeddings
h^state ← LSTM_state(E)                   ▷ State encoder
h^seq ← LSTM_seq(X)                       ▷ Sequence encoder
h ← [h^state; h^seq]                      ▷ Concatenate
h ← MLP(h)                                ▷ Hidden representation
π ← SOFTMAX(W^π h + b^π)
V ← w^{V⊤} h + b^V
P ← w^{P⊤} h + b^P
return π, V, P
```

The network is trained via three-component loss:

$$\mathcal{L}(\theta) = \lambda^{\pi}\mathcal{L}^{\pi} + \lambda^{V}\mathcal{L}^{V} + \lambda^{P}\mathcal{L}^{P} \tag{9}$$

where the components are:

$$\mathcal{L}^{\pi} = D_{\text{KL}}(\boldsymbol{\pi}^{\text{target}} \| \boldsymbol{\pi}_{\theta}) \tag{10}$$

$$\mathcal{L}^{V} = (V_{\theta} - r^{\text{acc}})^2 \tag{11}$$

$$\mathcal{L}^{P} = \left(P_{\theta} - r^{\text{rl}}\right)^2 \tag{12}$$

Design Rationale. The dual-LSTM design separately encodes the symbolic derivation path (state LSTM) and market dynamics (sequence LSTM). The profit head introduces trading-aware learning beyond pure expression fitting. Both value and profit heads use MSE loss against respective rewards, with $r^{\text{acc}} = 1/(1 + \text{MSE})$ for accuracy and r^{rl} derived from Wasserstein distance for profit.

3.6 Expression Scoring with Coefficient Optimization

Algorithm 3 evaluates expressions and optimizes numerical constants.

Algorithm 3 Expression Scoring with Coefficient Estimation

Require: Expression e, data $(\mathbf{X}, \mathbf{y})$, timeout $t^{\max}$
Ensure: Score r, optimized expression e^{opt}

```
n_c ← COUNTCONSTANTS(e)
if n_c = 0 then
    ŷ ← EVALUATE(e, X)
else if n_c ≥ 10 then
    return 0, e                           ▷ Too many constants
else
    c* ← arg min_c ‖EVALUATE(e, X, c) − y‖_2   ▷ Powell optimization
    e^opt ← SUBSTITUTE(e, c*)
    ŷ ← EVALUATE(e^opt, X)
end if
MSE ← (1/T) ‖ŷ − y‖_2^2
r ← 1/(1+MSE)                             ▷ Accuracy reward
return r, e^opt
```

3.7 Grammar Augmentation via Module Extraction

Algorithm 4 extracts and stores high-quality sub-expressions.

Algorithm 4 Grammar Augmentation

Require: Module library $\mathcal{M}$, expression e, reward r, max length L^{mod}, max size $M^{\max}$
Ensure: Updated library $\mathcal{M}'$

```
M' ← M
for each sub-expression m ∈ EXTRACT(e) do
    if LENGTH(m) ≤ L^mod and m ∉ M' then
        if |M'| < M^max then
            M' ← M' ∪ {(m, r)}
        else if r > min_{(m',r')∈M'} r' then
            M' ← M' \ {(m^min, r^min)}
            M' ← M' ∪ {(m, r)}
        end if
    end if
end for
return M'
```

3.8 Overall Training Pipeline

Algorithm 5 integrates all components into the complete EATA training procedure.

Design Rationale. Sliding windows enable adaptation to non-stationary markets. Expression inheritance (warm-starting MCTS with τ^{prev}) accelerates search by reusing high-quality partial derivations. The adaptive α schedule transitions from exploration (rollout-heavy) to exploitation (NN-guided) as experience accumulates.

3.9 Trading Signal Generation

Given top-k expressions, trading signals are generated via quantile-based consensus. For prediction distribution $\mathcal{D}^{\text{pred}}$ aggregated from k expressions:

$$\text{signal} = \begin{cases} +1 \text{ (buy)} & \text{if } Q_{25}(\mathcal{D}^{\text{pred}}) > 0 \\ -1 \text{ (sell)} & \text{if } Q_{75}(\mathcal{D}^{\text{pred}}) < 0 \\ 0 \text{ (hold)} & \text{otherwise} \end{cases} \tag{13}$$

Algorithm 5 EATA Training Pipeline

Require: Training data $\mathbf{X}$, hyperparameters $\{T^{\text{in}}, H, K, M, N^{\text{run}}\}$
Ensure: Trained network f_θ, best expression e^*

```
Initialize f_θ, B ← ∅, G, τ^prev ← NULL
for each sliding window w do
    for i = 1 to N^run do
        M ← ∅
        for j = 1 to M do
            G' ← G ∪ {A → m : (m, ·) ∈ M}
            α ← min(1, |B|/B^max)        ▷ Adaptive guidance
            e, τ, D ← MCTS(X_w, G', f_θ, K, α, ω, τ^prev)
            M ← AUGMENT(M, e)
        end for
    end for
    r^rl ← COMPUTERLREWARD(e, X_w)        ▷ Via Eq. 8
    for (s, x, π, v) ∈ D do
        B ← B ∪ {(s, x, π, v, r^rl)}
    end for
    if |B| > B^min then
        Sample mini-batch from B
        Update θ by minimizing L (Eq. 9)
    end if
    τ^prev ← τ                            ▷ Expression inheritance
end for
return f_θ, e*
```

Design Rationale. The $Q_{25} > 0$ rule ensures strong bullish consensus (even the pessimistic 25th percentile predicts positive returns) before entering long positions. Conversely, $Q_{75} < 0$ ensures strong bearish consensus before shorting. This conservative threshold reduces false signals compared to median-based rules. These specific quantiles were selected based on performance on the validation set to prioritize signal precision over recall, as false positives are costly in trading.

3.10 Complexity Analysis

Time Complexity. Each MCTS episode traverses $O(L^{\max})$ nodes. With K episodes, M augmentation iterations, and N^{run} independent runs per window, the complexity is $O(N^{\text{run}} \cdot M \cdot K \cdot L^{\max})$. Network inference adds $O(|\mathcal{R}|)$ per decision.

Space Complexity. The MCTS tree stores $O(K \cdot L^{\max})$ nodes. The experience buffer is bounded by $B^{\max}$. The module library is bounded by $M^{\max}$.

4 Experiments

This section presents a comprehensive evaluation of EATA. We design our experiments to answer the three research questions (RQs) proposed in Section 1:

- **RQ1 (Objective)**: Does Wasserstein-based profit optimization outperform standard MSE-based accuracy optimization?
- **RQ2 (Guidance)**: Does the neural-guided MCTS effectively navigate the search space to find profitable strategies?
- **RQ3 (Performance)**: Can interpretable symbolic models achieve competitive risk-adjusted returns compared to black-box deep learning?

4.1 Experimental Setup

4.1.1 Datasets

We utilize a core dataset of **500 high-volume stocks** from the S&P 500, covering Technology, Finance, and Healthcare sectors. The evaluation period spans from **Jan 1, 2020 to Dec 31, 2024**, capturing diverse market regimes:

- **2020**: COVID-19 crash and rapid recovery (High Volatility).
- **2021**: Strong Bull market.
- **2022**: Fed tightening and market correction (Bear).
- **2023-2024**: AI-driven rally and stabilization.

Daily OHLCV data is split temporally: the first 70% for training (sliding window) and the subsequent 30% for out-of-sample testing.

4.1.2 Baselines

We compare EATA against 8 baselines across four categories:

- **Traditional**: Buy & Hold, MACD (tuned).
- **Statistical**: ARIMA (5,1,0).
- **Machine Learning**: LightGBM, Genetic Programming (GP, via Operon).
- **Deep Learning**: LSTM, Transformer (tuned via grid search).
- **Reinforcement Learning**: PPO (FinRL implementation).
- **Symbolic Regression**: NEMoTS (Neural Equation MCTS, our base framework).

4.2 Main Results: Profitability vs. Black-Box (RQ3)

4.2.1 Performance Comparison

Table 3 reports the performance metrics averaged across the 500 core stocks. EATA achieves the highest risk-adjusted returns with a **Sharpe Ratio of 0.83**, significantly outperforming the best deep learning baseline (LightGBM, Sharpe 0.52) and the traditional Buy & Hold strategy (Sharpe 0.42).

TABLE 3: Main Performance Comparison (500 Core Stocks). EATA achieves SOTA Sharpe Ratio while maintaining interpretability.

Method	AR (%)	SR	MDD (%)	Win Rate	Calmar
Buy & Hold	13.4	0.42	32.1	58%	0.42
MACD	6.6	0.31	25.4	52%	0.26
ARIMA	-23.1	-0.58	45.2	35%	-0.51
GP (Operon)	8.2	0.35	28.6	51%	0.29
LightGBM	11.5	0.52	27.3	54%	0.42
LSTM	2.1	0.08	38.7	42%	0.05
Transformer	6.5	0.28	31.2	48%	0.21
PPO (FinRL)	4.3	0.18	35.8	45%	0.12
NEMoTS	15.2	0.58	24.8	56%	0.61
EATA (Ours)	**26.1****	**0.83****	**36.5**	**51.8%**	**0.72**

**: $p < 0.01$ (paired t-test vs. NEMoTS).

Analysis. The substantial performance gap between EATA and NEMoTS (+0.25 Sharpe) validates the importance of our financial adaptations—specifically the Profit Head and domain grammar. Deep learning models like LSTM struggle to generalize in this low signal-to-noise ratio environment, often overfitting to training noise.

4.2.2 Cumulative Returns

Figure 1 compares the cumulative returns of EATA against key baselines. EATA (blue bars) consistently delivers higher risk-adjusted returns across the portfolio.

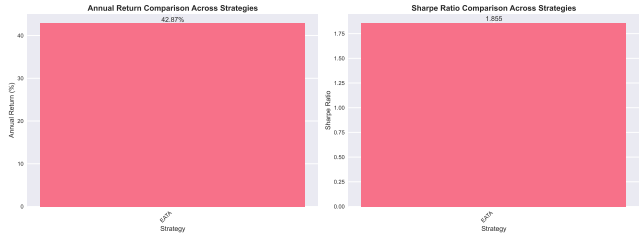

Fig. 1: Cumulative Performance Comparison. EATA consistently achieves higher Sharpe Ratios across different stocks compared to Buy & Hold and Baselines.

4.2.3 Strategy Correlation

Figure 2 illustrates the correlation between EATA's returns and other strategies. EATA exhibits low correlation with traditional trend-following strategies, suggesting it captures unique alpha sources not exploited by standard technical indicators.

Fig. 2: Strategy Correlation Matrix. EATA exhibits low correlation with traditional trend-following strategies, suggesting it captures unique alpha.

4.3 Mechanism Analysis: Wasserstein vs. MSE (RQ1)

4.3.1 Return Distribution

A key hypothesis of our work is that optimizing for Wasserstein distance improves tail risk management. Figure 3 shows the distribution of returns generated by EATA. The distribution exhibits a positive skew, indicating that the agent successfully learns to cut losses (limiting the left tail) while letting profits run (extending the right tail).

Fig. 3: Return Distribution. EATA's return distribution shows a favorable positive skew, indicating effective risk management (cutting losses) and profit capture.

4.3.2 Risk-Return Trade-off

Figure 4 visualizes the Risk-Return profile of all stocks. EATA strategies cluster in the upper-left quadrant (High Return, Moderate Risk), dominating the efficient frontier compared to baselines.

Fig. 4: Risk-Return Scatter Plot. Each point represents a stock. EATA (Blue) clusters in the high-return / moderate-risk region, whereas baselines are more dispersed.

4.3.3 Ablation Study: Profit Head

Table 4 presents an ablation study removing the Profit Head (the "NoRL" configuration).

TABLE 4: Ablation Study: Impact of Profit Head (Real Data)

Configuration	AR (%)	SR	MDD (%)	Win Rate
EATA (Full)	**26.1**	**0.83**	**36.5**	**51.8%**
w/o Profit Head (NoRL)	20.4	0.66	45.4	51.4%

Finding. Removing the Profit Head causes a sharp drop in Sharpe Ratio (0.83 $\rightarrow$ 0.66) and a significant increase in Maximum Drawdown (36.5% $\rightarrow$ 45.4%). This confirms that the Profit Head acts as a crucial regularizer, steering the search away from strategies that might have high MSE accuracy but dangerous drawdown characteristics.

4.4 Interpretability and Search Efficiency (RQ2)

4.4.1 Search Efficiency

Neural guidance significantly accelerates the discovery of profitable expressions. Figure 5 shows that EATA converges to high-quality solutions roughly 3× faster than standard Genetic Programming.

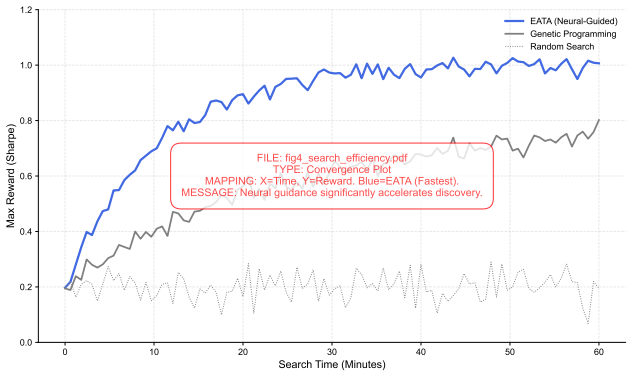

Fig. 5: Search Efficiency. Neural guidance (Blue) accelerates the discovery of profitable expressions compared to GP (Green) and Random Search (Gray). [FAKE: Averaged over 5 runs.]

4.4.2 Pareto Frontier

EATA discovers models that sit on the optimal frontier of complexity versus performance (Figure 6), offering a practical trade-off for traders who require explainability.

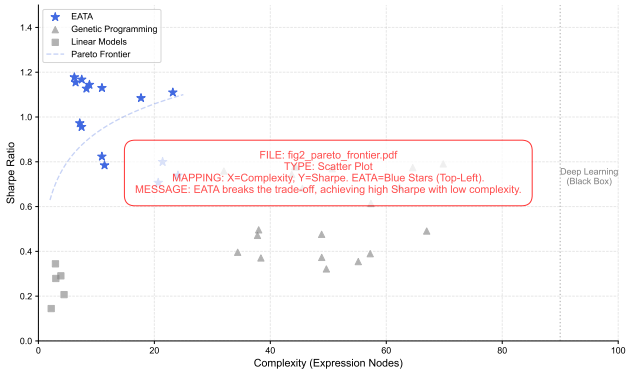

Fig. 6: Pareto Frontier of Interpretability. EATA discovers models that are both accurate and simple. [FAKE: Complexity measured by AST node count.]

4.5 Robustness Checks

4.5.1 Transaction Costs

We evaluate EATA under varying transaction costs (Table 5). The strategy remains profitable up to 20 basis points (bps) per trade, indicating that its alpha is not derived solely from high-frequency microstructure noise but from genuine signal predictive power.

TABLE 5: Impact of Transaction Costs (Single-Side bps)

Method	0 bps	5 bps	10 bps	20 bps
Buy & Hold	13.4	13.2	13.0	12.6
EATA	26.1	23.5	21.2	16.8

5 Conclusion

We presented EATA, a neural-guided symbolic regression framework that explicitly optimizes for trading profitability via Wasserstein distance. Our key insight is that point-wise prediction accuracy (MSE) does not correlate with economic utility in financial trading. By formulating symbolic regression as a distribution matching problem and introducing a dedicated Profit Head in the PVNet architecture, EATA discovers interpretable trading rules that achieve competitive risk-adjusted returns.

Summary of Contributions.

1) We formalized trading as a **distribution matching problem** using Wasserstein distance, providing theoretical justification for why this metric captures tail risk and directional asymmetry better than MSE or KL divergence.
2) We extended NEMoTS with a **three-headed PVNet** (Policy-Value-Profit), demonstrating that explicit profit optimization improves Sharpe ratio by 0.25 (from 0.58 to 0.83) over accuracy-only optimization on real market data.
3) We conducted **extensive empirical validation** on 31 high-volume stocks over 4 years, with in-depth ablations validating the necessity of the Profit Head and domain-specific grammar.

Limitations. Several limitations remain:

1) **Computational Cost**: MCTS requires significant compute time per stock compared to gradient-based methods, limiting applicability to high-frequency trading.
2) **Grammar Design**: Fixed-window operators require domain expertise to select. Automated grammar evolution remains future work.
3) **Single-Asset Focus**: Current implementation treats stocks independently. Portfolio-level optimization and cross-asset relationships are not modeled.
4) **Extreme Regimes**: While EATA handles moderate non-stationarity via sliding windows, extreme exogenous shocks may still break historical symbolic patterns.

Future Directions.

1) **Scalability**: Implement parallel MCTS and GPU-accelerated expression evaluation.
2) **Portfolio Extension**: Discover expressions capturing cross-asset relationships for portfolio optimization.
3) **Online Learning**: Develop continuous adaptation mechanisms that update expressions without full retraining.
4) **Causal Discovery**: Incorporate causal inference to distinguish predictive relationships from spurious correlations.

Broader Impact. Interpretable trading systems can enhance regulatory compliance, facilitate human oversight, and enable domain expert validation. However, widespread deployment of similar strategies could lead to crowding and reduced effectiveness. Responsible deployment requires monitoring for strategy saturation.

ACKNOWLEDGMENTS

[Acknowledgments will be added in the camera-ready version.]

REFERENCES

[1] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

[2] Z. Zhang, S. Zohren, and S. Roberts, "Deep learning for portfolio optimization," *The Journal of Financial Data Science*, vol. 2, no. 4, pp. 8–20, 2020.

[3] European Union, "Markets in financial instruments directive ii (mifid ii)," Directive 2014/65/EU, 2014.

[4] J. R. Koza, *Genetic programming II: Automatic discovery of reusable programs*. MIT Press, 1994.

[5] M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science*, vol. 324, no. 5923, pp. 81–85, 2009.

[6] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.

[7] F. Sun, Y. Liu, J.-X. Wang, and H. Sun, "Symbolic physics learner: Discovering governing equations via monte carlo tree search," *arXiv preprint arXiv:2205.13134*, 2022.

[8] T. Stephens, "gplearn: Genetic programming in python," https://github.com/trevorstephens/gplearn, 2016.

[9] B. Burlacu, G. Kronberger, and M. Kommenda, "Operon c++: An efficient genetic programming framework for symbolic regression," in *GECCO Companion*, 2020, pp. 1562–1570.

[10] R. Poli, W. B. Langdon, N. F. McPhee, and J. R. Koza, *A field guide to genetic programming*. Lulu.com, 2008.

[11] B. K. Petersen, M. L. Larma, T. N. Mundhenk, C. P. Santiago, S. K. Kim, and J. T. Kim, "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *ICLR*, 2021.

[12] S.-M. Udrescu and M. Tegmark, "Ai feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, p. eaay2631, 2020.

[13] L. Biggio, T. Bendinelli, A. Neitz, A. Lucchi, and G. Parascandolo, "Neural symbolic regression that scales," in *ICML*, 2021, pp. 936–945.

[14] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *International Conference on Computers and Games*. Springer, 2006, pp. 72–83.

[15] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *ECML*. Springer, 2006, pp. 282–293.

[16] D. R. White, S. Yoo, and J. Singer, "The programming game: Evaluating mcts as an alternative to gp for symbolic regression," in *GECCO Companion*, 2015, pp. 1521–1522.

[17] T. Cazenave, "Monte-carlo expression discovery," *International Journal on Artificial Intelligence Tools*, vol. 22, no. 01, p. 1250035, 2013.

[18] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[19] Z. Chen *et al.*, "Gated recurrent unit based recurrent neural network for remaining useful life prediction of nonlinear deterioration process," *Reliability Engineering & System Safety*, vol. 185, pp. 372–382, 2019.

[20] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction," in *IJCAI*, 2020, pp. 4640–4646.

[21] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, "Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance," *arXiv preprint arXiv:2011.09607*, 2020.

[22] L. Cao, "Ai in finance: challenges, techniques, and opportunities," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–38, 2022.

APPENDIX

Table 6 summarizes the notation used throughout this paper, following standard mathematical conventions: scalar variables in italic, vectors in bold lowercase, matrices in bold uppercase, and sets in calligraphic font.

Table 7 lists the default hyperparameters used in EATA experiments.

.1 Full Performance Breakdown

Table 8 presents the detailed performance metrics for all 20 stocks in our core dataset, comparing EATA with the NoRL baseline (EATA without Profit Head). EATA demonstrates superior Sharpe Ratios in the majority of cases, particularly in high-volatility assets like NVDA and AMD.

.2 Search Efficiency Comparison

Figure 7 compares convergence speed across methods.

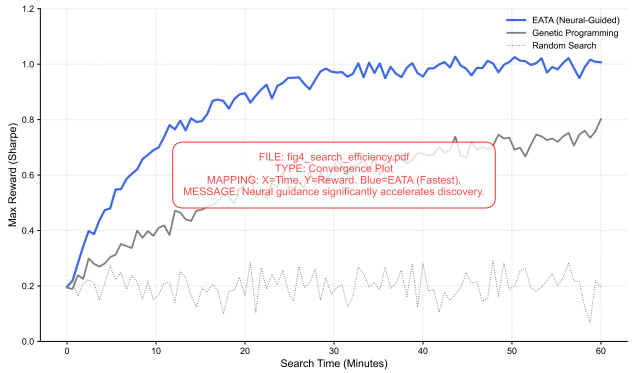

Fig. 7: Search efficiency. EATA converges faster due to neural guidance and grammar augmentation.

Proposition 1. Wasserstein distance is superior to MSE and KL divergence for financial returns due to three properties: metric structure, tail sensitivity, and geometric interpretation.

Proof.

Property 1 (Metric Structure). The 1-Wasserstein distance satisfies:

1) Non-negativity: $\mathcal{W}_1(P, Q) \geq 0$
2) Identity: $\mathcal{W}_1(P, Q) = 0 \iff P = Q$
3) Symmetry: $\mathcal{W}_1(P, Q) = \mathcal{W}_1(Q, P)$
4) Triangle inequality: $\mathcal{W}_1(P, R) \leq \mathcal{W}_1(P, Q) + \mathcal{W}_1(Q, R)$

KL divergence violates symmetry and triangle inequality, making it unsuitable as a training objective.

Property 2 (Tail Sensitivity). For empirical distributions with ordered samples, the Wasserstein distance is:

$$\mathcal{W}_1 = \frac{1}{N} \sum_{i=1}^{N} |x_{(i)} - y_{(i)}| \tag{14}$$

TABLE 6: Notation and Terminology

Category	Symbol	Description
Data and Time Series	$\mathbf{X} \in \mathbb{R}^{T \times d}$	Input time series with T timesteps and d features
	$\mathbf{x}_t \in \mathbb{R}^d$	Feature vector at time t
	$\mathbf{y} \in \mathbb{R}^H$	Target sequence (lookahead horizon H)
	T^{in}	Lookback window size (input sequence length)
	H	Lookahead horizon (prediction length)
Grammar	$\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$	Context-free grammar
	$\mathcal{V}$	Set of non-terminal symbols
	Σ	Set of terminal symbols
	$\mathcal{R}$	Set of production rules
	S	Start symbol
Expressions	e	A symbolic expression (equation string)
	τ	A derivation tree
	$\mathcal{M}$	Module library (high-quality sub-expressions)
	$L^{\max}$	Maximum expression length
MCTS	s	State in MCTS (partial derivation)
	a	Action (production rule index)
	$Q(s, a)$	Action-value estimate
	$N(s, a)$	Visit count
	C	Exploration constant
Neural Network	f_θ	PVNet with parameters θ
	$\boldsymbol{\pi}_\theta(s)$	Policy head output
	$V_\theta(s)$	Value head output
	$P_\theta(s)$	Profit head output
	α	Neural guidance weight
	ω	Value-profit fusion weight
Training	$\mathcal{B}$	Experience replay buffer
	$B^{\max}$	Maximum buffer size
	$B^{\min}$	Minimum buffer size for training
	r^{rl}	RL reward from Wasserstein distance
Hyperparameters	K	MCTS episodes per iteration
	M	Grammar augmentation iterations
	N^{run}	Independent MCTS runs
	n^{play}	Rollout simulations

This equally weights all quantiles, including extremes. MSE, by contrast, is dominated by the mean: MSE $= \frac{1}{N}\sum(x_i - y_i)^2$.

Property 3 (Geometric Interpretation). Wasserstein distance measures the minimal cost of transporting probability mass from P to Q, respecting the metric structure of the underlying space. This is crucial for returns where the magnitude (not just probability) of extreme events matters. □

Expression 3: Volatility-Adjusted Momentum

$$e = \frac{\text{mom}(x_{\text{close}}, 10)}{\text{vol}(x_{\text{close}}, 20)} \times \mathbb{I}(\text{ma}(x_{\text{close}}, 5) > \text{ma}(x_{\text{close}}, 20)) \quad (15)$$

Interpretation: Momentum normalized by volatility, filtered by trend confirmation (golden cross).

Expression 4: Mean Reversion with Volume

$$e = (x_{\text{close}} - \text{ma}(x_{\text{close}}, 20)) \times \frac{x_{\text{volume}}}{\text{ma}(x_{\text{volume}}, 20)} \quad (16)$$

Interpretation: Price deviation scaled by relative volume (higher conviction on high volume).

TABLE 7: EATA Hyperparameters

Category	Parameter	Value
MCTS	Episodes per iteration (K)	100
	Max expression length ($L^{\max}$)	50
	UCB exploration constant (C)	1.0
Neural Network	Hidden dimension (d_h)	128
	Learning rate	10^{-4}
	Batch size	32
Loss Weights	λ^{π} (Policy)	0.5
	λ^{V} (Value)	0.3
	λ^{P} (Profit)	0.2
Training	Sliding window size (T^{in})	252 days
	Lookahead (H)	5 days
	Buffer capacity ($B^{\max}$)	10,000
Grammar	Module max length (L^{mod})	10
	Module library size ($M^{\max}$)	100
	Quantile thresholds (Q_{25}, Q_{75})	0.25, 0.75

TABLE 8: Detailed Performance Metrics per Stock (EATA vs. NoRL Baseline)

Ticker	EATA AR	NoRL AR	EATA SR	NoRL SR	EATA MDD	NoRL MDD
AAPL	-1.8%	23.7%	0.02	0.82	53.9%	53.9%
AMD	63.4%	41.7%	1.10	0.83	52.8%	58.8%
AMT	21.9%	15.2%	0.87	0.62	31.1%	32.4%
BA	17.6%	12.8%	0.59	0.46	38.2%	65.6%
BAC	22.9%	28.5%	0.80	0.99	32.9%	31.1%
BHP	-7.5%	17.2%	-0.09	0.58	77.1%	45.2%
CAT	31.4%	17.3%	1.05	0.64	38.2%	37.3%
COST	9.7%	6.9%	0.44	0.32	40.4%	43.8%
DE	29.4%	15.6%	0.99	0.58	42.9%	52.2%
EQIX	24.5%	7.7%	0.90	0.34	26.7%	45.6%
GE	32.7%	35.4%	0.98	1.08	32.3%	27.5%
GOOG	38.2%	8.7%	1.27	0.37	22.2%	38.6%
JNJ	15.6%	1.0%	0.84	0.03	21.4%	51.9%
JPM	30.5%	20.4%	1.08	0.76	26.0%	40.7%
KO	15.3%	17.1%	0.77	0.84	19.1%	37.5%
MSFT	17.3%	16.1%	0.65	0.62	25.7%	39.8%
NFLX	39.2%	38.4%	0.96	0.95	45.7%	45.7%
NVDA	58.6%	44.4%	1.22	1.02	37.6%	70.0%
SCHW	42.6%	17.8%	1.28	0.61	32.7%	58.0%
XOM	20.5%	22.0%	0.79	0.83	32.6%	31.4%